

3_gyakorlat_2026_tavasz

1. feladat

Egy utazó ügynök az alábbi 10 városban járt januárban: Vac, Pecs, Szeged, Tata, Vac, Gyor, Pecs, Tata, Vac, Eger

Válogassuk ki a városokat egy új tömbbe úgy, hogy mindegyik név csak egyszer szerepeljen benne!

```
#include <iostream>
#include <string>
using namespace std;
int main()
{
    // Egy utazó ügynök az alábbi 10 városban járt januárban : Vac, Pecs,Szeged, Tata, Vac, Gyor, Pecs, Tata,
    // Vac, Eger
    // Válogassuk ki a városokat egy új tömbbe úgy hogy mindegyik név csak egyszer szerepeljen benne!
    string varosok[10] = { "Vac", "Pecs", "Szeged", "Tata", "Vac", "Gyor", "Pecs", "Tata", "Vac", "Eger" };
    string uj[10];
    int ujd = 0; // az új tömb darabszáma- jelenleg 0 üres a tömb
    for (int i = 0; i < 10; i++) // vegyük egyenként a városokat
    {
        int j = 0;
        while (j < ujd && varosok[i] != uj[j]) // döntjük el, hogy az adott város benne van-e már az új tömbben
        {
            j++;
        }
        if (j == ujd) // ha úgy szálltunk ki a ciklusból, hogy nem volt az adott város az új tömbben
        {
            uj[ujd] = varosok[i];
            ujd++; // növeljük az új tömbben lévő városok számát
        }
    }
    for (int i = 0; i < ujd; i++)
    {
        cout << uj[i] << " ";
    }
    return 0;
}
```

2. feladat

A Pontszam 8 elemű tömbben egy felmérő pontszámait helyeztük el (0-100) növekvő sorrendben. int Pontszam[8]={2,67,78,89,90,92,95,100};

Határozzuk meg, hogy egy billentyűzetről bekért pontszám eleme -e a tömbnek és ha igen hányadik? - LOGARITMIKUS KERESÉS

```
#include <iostream>
using namespace std;
int main( )
{
    int Pontszam[8]={2,67,78,89,90,92,95,100};
    int pont,k, A=0, F=7;
    cout<<"Adja meg a keresett pontot!";
    cin>>pont;

    do
    {
        k=(A+F)/2;
        if(Pontszam[k]<pont){A=k+1;}
        if(Pontszam[k]>pont){F=k-1;}
    }
    while(A<=F&&Pontszam[k]!=pont);
    if(A<=F)cout<<"A keresett pont a Pontszam tomb "<<k<<" . indexu eleme"; //0 az első elem
    else cout<<"A keresett pont nincs a Pontszam tombben";
    cout<<endl;
    return 0;
}
```

3. feladat

Állítson elő véletlenszerűen 100 számot a 2 – 500 zárt intervallumból (a 2 és az 500 is számít) úgy, hogy minden szám csak egyszer fordulhat elő. Úgy kell a feladatot megoldania, hogy minden szám egyenlő valószínűséggel szerepelhessen a kigenerált számok között. Az előállított számok közül válogassa ki a prímszámokat és állítsa elő a kigenerált számok közötti prímszámok négyzet összegét!

```
#include <iostream>
#include <ctime>
using namespace std;
int main()
{
    srand((unsigned int)time(NULL));
    int tmb[100];
    int ossz = 0;
    // 100 különböző szám kigenerálása
    cout << "A 100 kulonbozo szam : " << endl;
    int j = 0;
    while (j < 100)
    {
        int szam = rand() % 499 + 2;
        int k = 0;
        while (k < j && tmb[k] != szam)
        {
            k++;
        }
        if (k == j)
        {
            tmb[j++] = szam;
            cout << szam << " ";
        }
    }
    // prímszámok négyzetösszege
    cout << "\nA primszamok : " << endl;
    for (int i = 0; i < 100; i++)
    {
        int k = 2;
        int z = sqrt(tmb[i]);
        while (k <= z && tmb[i] % k != 0) // eldöntés tétele - eldöntjük, hogy a szám prím-e
        {
            k++;
        }
        if (k > z)
        {
            cout << tmb[i] << " ";
            ossz = ossz + tmb[i] * tmb[i]; //összegzés
        }
    }
    cout << "\nA szamok negyzetosszege : " << ossz;
    return 0;
}
```

4. feladat

A szotar.txt állományban legfeljebb 150 magyar szót tárolunk abc sorrendben. Minden szó új sorban helyezkedik el.

Hajtsa végre az alábbi feladatokat függvények segítségével:

1. Olvassa be a szavakat egy tömbbe és írja ki a szavak számát!
A **Beolvas()** függvény két paraméterrel rendelkezzen (a feltöltendő tömb és a megnyitandó állomány nevével és adja vissza a szótárban lévő szavak számát!)
2. **Leghosszabb()** adja vissza a leghosszabb szó értékét!

3. Keressen meg a tömbbe egy billentyűzetről bekért szót és írja ki a megtalált szó sorszámát! Használjon Lineáris keresést!
Linearis() - a függvény adja vissza a megtalált szó sorszámát úgy, hogy 0-t adjon vissza, ha nincs meg a keresett szót különben pedig a szó sorszámát (a sorszámok 1-el kezdődnek)
4. Keressen meg a tömbbe egy billentyűzetről bekért szót és írja ki a megtalált szósorszámát! Használjon Logaritmikus keresést!
Logaritmikus() - a függvény adja vissza a megtalált szó sorszámát úgy, hogy 0-t adjon vissza, ha nincs meg a keresett szót különben pedig a szó sorszámát (a sorszámok 1-el kezdődnek)
5. Döntse el van-e olyan szó a szótárban, amiben több a magánhangzó, mint a mássalhangzó! **bool Maganhangzo()**

```
#include <iostream>
#include <fstream>
#include <iomanip>
using namespace std;
int Beolvas(string *tmb, string fnev);
string Leghosszabb(string* tmb, int db);
int Linearis(string* tmb,int db, string szo);
int Logaritmikus(string* tmb,int db, string szo);
bool Maganhangzo(string* tmb, int db);
int main(int argc,int db, char* argv[])
{
    string szavak[150];
    int szavakszama = 0;
    string filenev = "szotar.txt";
    /* 1. Olvassa be a szotar.txt állományból a szavakat, és a következő feladatok megoldása
    során ezekkel dolgozzon!*/
    szavakszama = Beolvas(szavak, filenev);
    cout<<"A szavak szama "<< szavakszama<<endl;
    cout << "A szotar leghosszabb szava: " << Leghosszabb(szavak, szavakszama)<<endl;

    string szoveg;
    cout << "Adjon meg egy szot " << endl;
    cin >> szoveg;
    int sorszam= Linearis(szavak, szavakszama, szoveg);
    if (sorszam) cout << "A szotarban a keresett szo a: " << sorszam << ".helyen van" << endl;
    else cout << "A szotarban nincs benne a keresett szo"<< endl;

    sorszam = Logaritmikus(szavak, szavakszama, szoveg);
    if (sorszam) cout << "A szotarban a keresett szo a: " << sorszam << ".helyen van" << endl;
    else cout << "A szotarban nincs benne a keresett szo" << endl;

    if (Maganhangzo(szavak, szavakszama)) cout << "Van ilyen szo szotarban" << endl;
    else cout << "Nincs ilyen szo szotarban" << endl;
    return 0;
}
int Beolvas(string* tmb, string fnev)
{
    ifstream be(fnev);
    if (be.fail()) { cout << "Hibas fajlmagnyitas"; system("pause"); exit(1); }
    int db = 0;
    while (!be.eof())
    {
        be >> tmb[db];
```

```

        cout << tmb[db] << endl;
        db++;
    }
    be.close();
return db;
}
string Leghosszabb(string* tmb, int x)
{
    int leghossz = 0;
    for (int i = 0; i < x; i++)
    {
        if (tmb[i].size() > tmb[leghossz].size())
        {
            leghossz = i;
        }
    }
return tmb[leghossz];
}
int Linearis(string* tmb, int db, string szo)
{
    int i = 0;
    while (i < db && tmb[i] != szo)
    {
        i++;
    }
    if (i < db) return i + 1;
    else return 0;
}
int Logaritmikus(string* tmb, int db, string szo)
{
    int kozep, AH = 0, FH = db - 1;
    do
    {
        kozep = (AH + FH) / 2;
        if (tmb[kozep] < szo) AH = kozep + 1;
        if (tmb[kozep] > szo) FH = kozep - 1;
    } while (AH <= FH && tmb[kozep] != szo);
    if (AH <= FH)
    {
        return kozep + 1;
    }
    else
    {
        return 0;
    }
}
bool Maganhangzo(string* tmb, int db)
{
    int i = 0;
    while (i < db)
    {
        int magandb = 0;
        for (int j = 0; j < tmb[i].size(); j++)
        {
            if (tmb[i][j] == 'a' || tmb[i][j] == 'e' || tmb[i][j] == 'i' || tmb[i][j] == 'o' || tmb[i][j] == 'u')
                magandb++;
        }
    }
}

```

```
    }  
    if (tmb[i].size() / 2 < magandb)  
    {  
        cout << tmb[i] << endl;  
        return true;  
    }  
    i++;  
}  
return false;  
}
```